Python Data Types — Complete Comparison Guide

A clear revision sheet for understanding the similarities, differences, mutability, ordering, indexing, duplicates, and common operations of Python's core built-in data types.

1. The Big Picture

Python data types tell Python what kind of value an object represents and what operations can be performed on it.

Category	Data Type	Example	Main Purpose
Numeric	**int**	25	Whole numbers
Numeric	**float**	25.5	Decimal numbers
Numeric	**complex**	3 + 4j	Complex numbers
Text	**str**	"Python"	Characters / text
Sequence	**list**	[1, 2, 3]	Ordered, changeable collection
Sequence	**tuple**	(1, 2, 3)	Ordered, fixed collection
Mapping	**dict**	{"a": 1}	Key-value data
Set	**set**	{1, 2, 3}	Unique values
Boolean	**bool**	True / False	Logical truth values
Null	**NoneType**	None	Represents absence of a value

2. The Most Important Comparison: Collection Types

When comparing **str, list, tuple, set, and dict**, focus on six questions: **Is it ordered? Is it mutable? Are duplicates allowed? Does it use indexing? How are values accessed? What is it best used for?**

Type	Ordered?	Mutable?	Duplicates?	Indexing?	Access	Typical Use
str	Yes	No	Yes	Yes	Index / slice	Text
list	Yes	Yes	Yes	Yes	Index / slice	Changeable collection
tuple	Yes	No	Yes	Yes	Index / slice	Fixed collection
set	No fixed order	Yes	No	No	Membership / set operations	Unique values
dict	Insertion order preserved	Yes	Keys: no; values: yes	No numeric indexing	Key	Key-value data

3. Numeric Types: int vs float vs complex

Feature	int	float	complex
Example	10	10.5	3 + 4j
Represents	Whole numbers	Decimal / floating-point numbers	Real + imaginary parts
Indexing	No	No	No
Mutable?	No — numeric objects are immutable	No — numeric objects are immutable	No — numeric objects are immutable
Common conversions	int(), float()	int(), float()	complex()

4. Mutable vs Immutable

Mutable means the object can be changed after creation. **Immutable** means the object itself cannot be changed after creation.

Type	Mutable?	Example	What this means
int	No	10	Cannot change the integer object itself
float	No	10.5	Cannot change the float object itself
complex	No	2 + 3j	Cannot change the complex object itself
bool	No	True	Boolean objects are immutable
str	No	"Python"	Characters cannot be directly replaced
tuple	No	(1, 2, 3)	Elements cannot be reassigned
list	Yes	[1, 2, 3]	Elements can be changed, added, or removed
set	Yes	{1, 2, 3}	Elements can be added/removed
dict	Yes	{"a": 1}	Pairs can be added, updated, or removed
NoneType	No	None	Represents one singleton null value

5. List vs Tuple — Most Important Collection Difference

These two are frequently confused because both are ordered sequences that support indexing and slicing.

Feature	List	Tuple
Syntax	[1, 2, 3]	(1, 2, 3)
Ordered	Yes	Yes
Mutable	Yes	No
Duplicates	Allowed	Allowed
Indexing	Yes	Yes
Slicing	Yes	Yes
Add/remove items	Yes	No direct item modification
Typical use	Data that changes	Fixed/record-like data

6. Set vs Dictionary — Curly Braces

Both use curly braces, but their internal structure is different.

Feature	Set	Dictionary
Syntax	{1, 2, 3}	{"name": "Shrishti"}
Stores	Unique values	Key-value pairs
Duplicates	Not allowed	Keys: not allowed; values: allowed
Indexing	No	No numeric indexing; access by key
Mutable	Yes	Yes
Empty form	set()	{}
Typical use	Unique items / set math	Structured key-value data

7. Indexing, Slicing and Access

Type	Indexing	Slicing	How values are accessed
str	Yes	Yes	Position
list	Yes	Yes	Position
tuple	Yes	Yes	Position
set	No	No	Membership / iteration
dict	No numeric indexing	No	Key
numbers	No	No	The value itself

8. Duplicates, Ordering and Access — Quick Decision Table

If you need...	Choose	Why?
Text	str	Sequence of characters
An ordered collection you will modify	list	Mutable + indexed
An ordered collection that should not change	tuple	Immutable + indexed
Only unique values	set	Duplicates are removed
Fast membership/set mathematics	set	Designed for set operations and membership
Data identified by meaningful labels	dict	Access using keys
Whole numbers	int	Integer numeric type
Decimal calculations	float	Floating-point numeric type
Real + imaginary values	complex	Complex numeric type
True/False state	bool	Boolean truth values
No value / missing value	None	Represents absence of a value

9. Common Similarities

• **All are Python objects.** You can use type() to inspect their type.

• **Variables can refer to any of them.** Python is dynamically typed, so a variable name is not permanently restricted to one data type.

• **Most can be assigned, passed to functions, returned from functions, and stored inside other collections.**

• **Several collection types support iteration.** Strings, lists, tuples, sets, and dictionaries can all be iterated over.

• **Several types support membership testing with in.** The exact meaning depends on the type; for a dictionary, direct membership checks keys.

10. Common Differences to Memorize

• **Mutable vs immutable:** list, set, and dict are mutable; str and tuple are immutable; numeric, bool, and NoneType objects are immutable.

• **Ordered vs unordered:** str, list, and tuple are ordered sequences. Sets do not provide positional ordering. Dictionaries preserve insertion order in modern Python but are accessed by keys, not numeric indexes.

• **Duplicates:** lists, tuples, and strings can contain repeated values/characters. Sets do not. Dictionary keys must be unique, while dictionary values may repeat.

• **Access:** sequences use positions; dictionaries use keys; sets are primarily accessed through membership and iteration.

• **Purpose:** choose the data type based on the problem rather than simply on syntax.

11. Mini Examples — Same Data, Different Types

The same three values can be represented differently depending on what behavior you need:

Type	Code	Meaning
list	[10, 20, 20]	Keep order and duplicates; can modify
tuple	(10, 20, 20)	Keep order and duplicates; cannot modify
set	{10, 20, 20}	Duplicates collapse to unique values
str	"102020"	Treat the data as text
dict	{"a": 10, "b": 20}	Give each value a meaningful key

12. Final Memory Map

String → text
List → ordered + mutable + duplicates
Tuple → ordered + immutable + duplicates
Set → unique values + mutable + no indexing
Dictionary → key → value + mutable
int → whole number
float → decimal number
complex → real + imaginary
bool → True / False
None → absence of a value

Best interview shortcut: When comparing list, tuple, set, and dictionary, remember **Order** → **Mutability** → **Duplicates** → **Indexing** → **Access method** → **Use case.**